

PROIECTARE SOFTWARE

TEMA 1 - PROBLEMA 15

Agentie Imobiliara - Gestiune Inchirieri

Student:	Pruteanu Marius
Grupa:	30235
Facultatea:	Facultatea de Automatica si Calculatoare
Universitatea:	Universitatea Tehnica din Cluj-Napoca
An universitar:	2025 - 2026
Termen predare:	23 - 25 martie 2026
Email predare:	anca.iordan@cs.utcluj.ro

1. Enuntul Problemei

Problema 15 solicita dezvoltarea unei aplicatii software pentru o agentie imobiliara, destinata administrarii inchirierilor de locuinte. Aplicatia trebuie sa permita:

- Adaugarea, stergerea si actualizarea informatiilor despre locuintele disponibile pentru inchiriat;
- Vizualizarea listei locuintelor disponibile sortata dupa locatie si pret, cu afisarea imaginilor aferente;
- Filtrarea listei locuintelor dupa locatie, pret, tip locuinta sau numar de camere;
- Adaugarea, stergerea si actualizarea informatiilor despre locuintele inchiriate, inclusiv informatii despre clienti;
- Cautarea locuintelor inchiriate de un anumit client.

Aplicatia utilizeaza arhitectura Model-View-Presenter (MVP) si o baza de date relationala MySQL pentru persistenta datelor.

2. Instrumente Utilizate

Limbaj de programare:	Java 17
Framework UI:	JavaFX 17
Baza de date:	MySQL 8.0
IDE:	IntelliJ IDEA
Build tool:	Apache Maven
Driver JDBC:	mysql-connector-java 8.0
Diagrame UML:	draw.io (diagrams.net)
Arhitectura:	Model-View-Presenter (MVP)

3. Justificarea Limbajului de Programare Ales

Java a fost ales din urmatoarele motive:

- Portabilitate: aplicatia ruleaza pe orice sistem de operare cu JVM, fara modificari ale codului;
- JavaFX ofera componente grafice desktop moderne (TabPane, TableView, ImageView, DatePicker) necesare aplicatiei;
- Integrare nativa cu MySQL prin JDBC (mysql-connector-java), cu suport complet pentru PreparedStatement care previne SQL Injection;
- OOP avansat: interfetele, clasele abstracte si injectia de dependente prin constructor faciliteaza implementarea curata a arhitecturii MVP si respectarea principiilor SOLID;
- Garbage Collector automat elimina erorile de tip memory leak frecvente in C++;
- Ecosistem matur si documentatie abundenta, util in contextul academic.

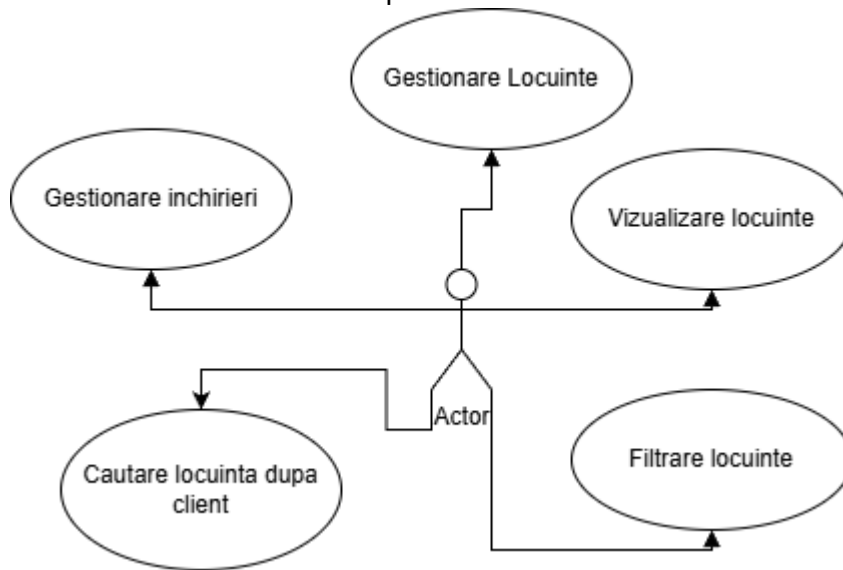
4. Descrierea Diagramelor UML

4.1 Diagrama Cazurilor de Utilizare

Diagramei prezinta un singur actor: Agentul imobiliar, care are acces la toate functionalitatile sistemului. Cazurile de utilizare identificate sunt:

- Gestionare locuinte: Adaugare, Stergere, Actualizare;
- Vizualizare lista locuinte (sortata dupa locatie si pret, cu imagini);
- Filtrare locuinte: dupa locatie, pret (interval min-max), tip locuinta, numar camere;
- Gestionare clienti: Adaugare, Stergere, Actualizare;
- Gestionare contracte de inchiriere: Adaugare, Stergere, Actualizare;

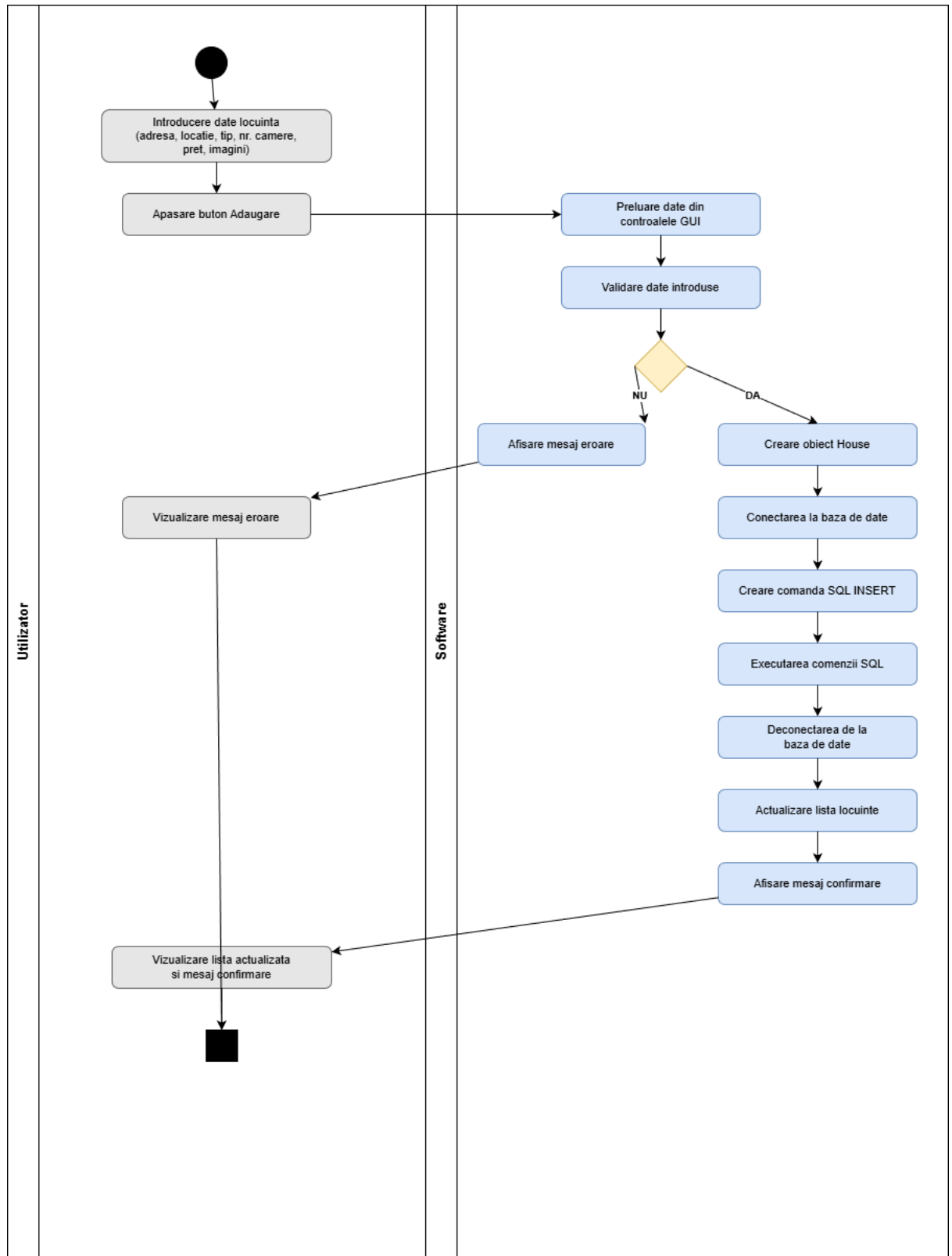
- Cautare locuinte inchiriate dupa numele unui client.

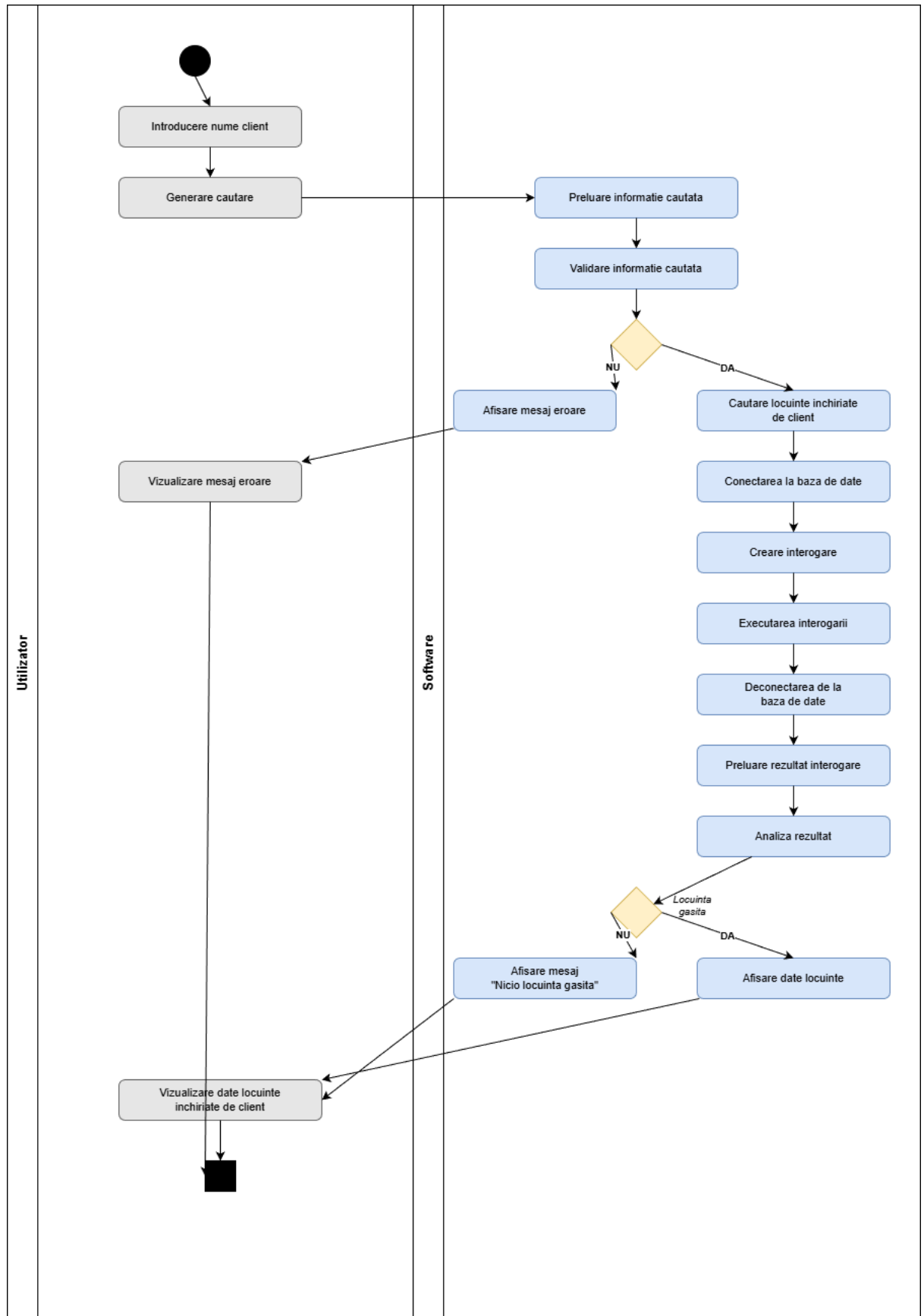


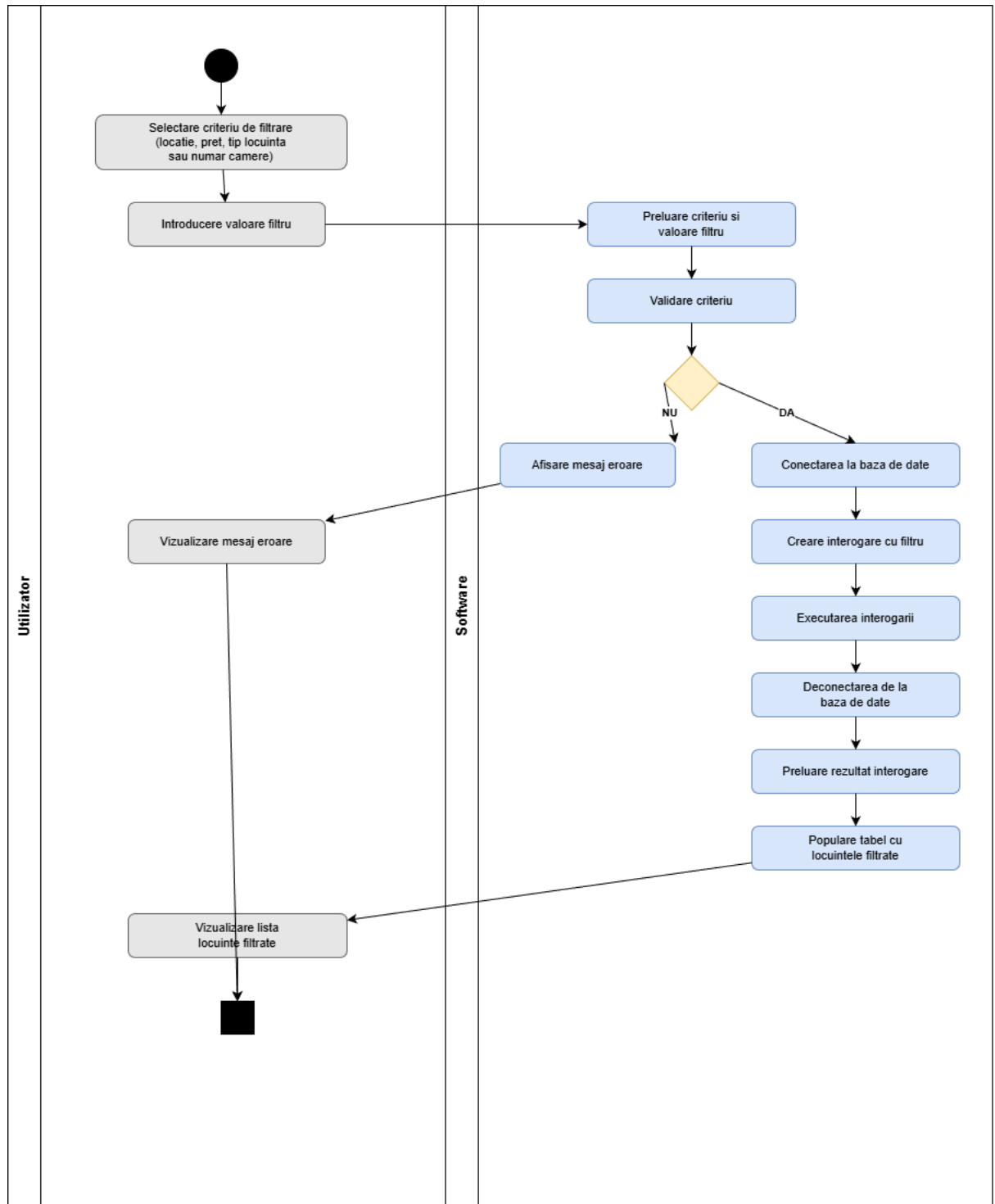
4.2 Diagrame de Activitati

S-au realizat diagrame de activitati pentru fiecare caz de utilizare, descriind fluxul complet cu ramificatii pentru succes si eroare:

- Adaugare locuinta: validare campuri -> INSERT -> confirmare/eroare -> refresh tabel;
- Stergere locuinta: selectie din tabel -> DELETE -> confirmare/eroare -> refresh tabel;
- Actualizare locuinta: selectie -> modificare campuri -> validare -> UPDATE -> refresh;
- Filtrare locuinte: selectie criteriu -> introducere valoare -> query filtrat -> afisare;
- Adaugare contract: validare ID-uri (verificare existenta locuinta si client in BD) -> verificare date -> INSERT -> marcare locuinta indisponibila -> refresh;
- Cautare dupa client: introducere nume -> cautare clienti -> extragere contracte -> afisare locuinte.







4.3 Diagrama de Pachete

Aplicatia este structurata in pachete conform MVP, cu dependente unidirectionale:

- org.example.model: House, Client, Rental, HouseType - clasele de domeniu;
- org.example.model.repository: interfetele IHouseRepository, IClientRepository, IRentalRepository, implementarile concrete si DatabaseConnection;
- org.example.presenter: IAgencyGUI (interfata pentru view) si AgencyPresenter (logica de coordonare);

- org.example.view: JavaFXGUI - implementeaza IAgencyGUI si gestioneaza interfata grafica;
- org.example: clasa Main, punctul de intrare in aplicatie.



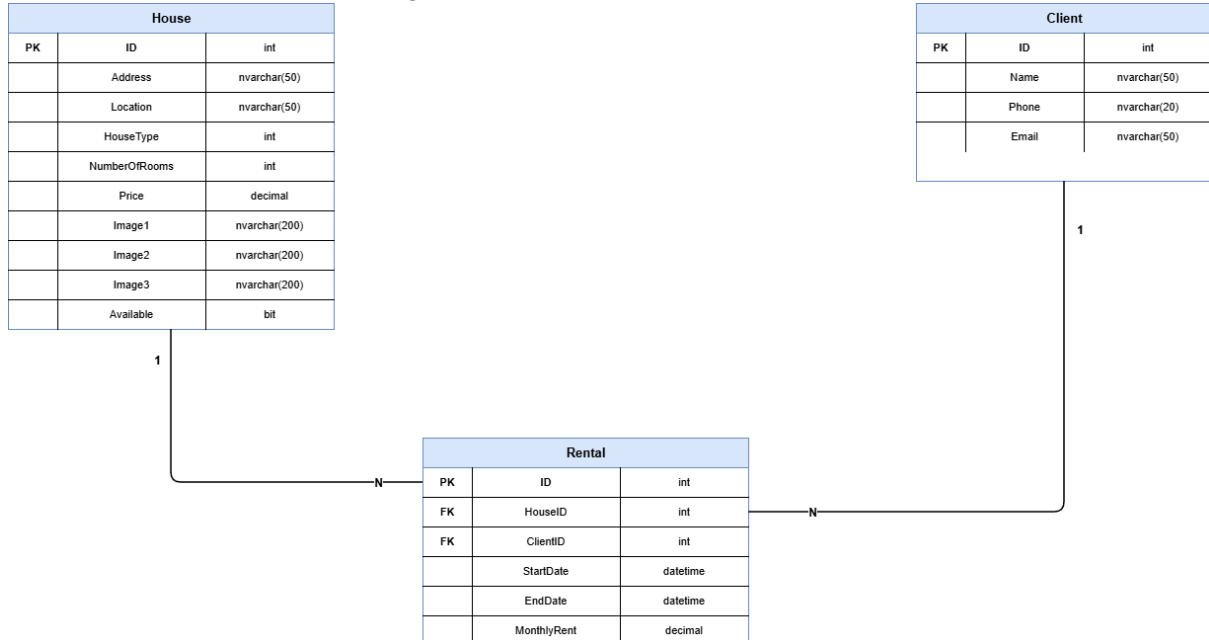
Dependentele respecta regula MVP: View depinde de presenter prin interfata IAgencyGUI, Presenter depinde de interfețele Model-ului, Model nu cunoaste View-ul.

4.4 Diagrama Entitate-Relatie (ER)

Baza de date RentalAgencyDB contine trei tabele:

- House (ID PK, Address, Location, HouseType INT, NumberOfRooms, Price, Image1, Image2, Image3, Available BOOLEAN);
- Client (ID PK, Name, Phone, Email);
- Rental (ID PK, HouseID FK->House.ID, ClientID FK->Client.ID, StartDate DATE, EndDate DATE nullable, MonthlyRent DOUBLE).

Relatii: un Client poate incheia oricati contracte Rental (1:N). O locuinta House poate aparea in mai multe contracte de-a lungul timpului (1:N). EndDate NULL semnifica un contract activ.



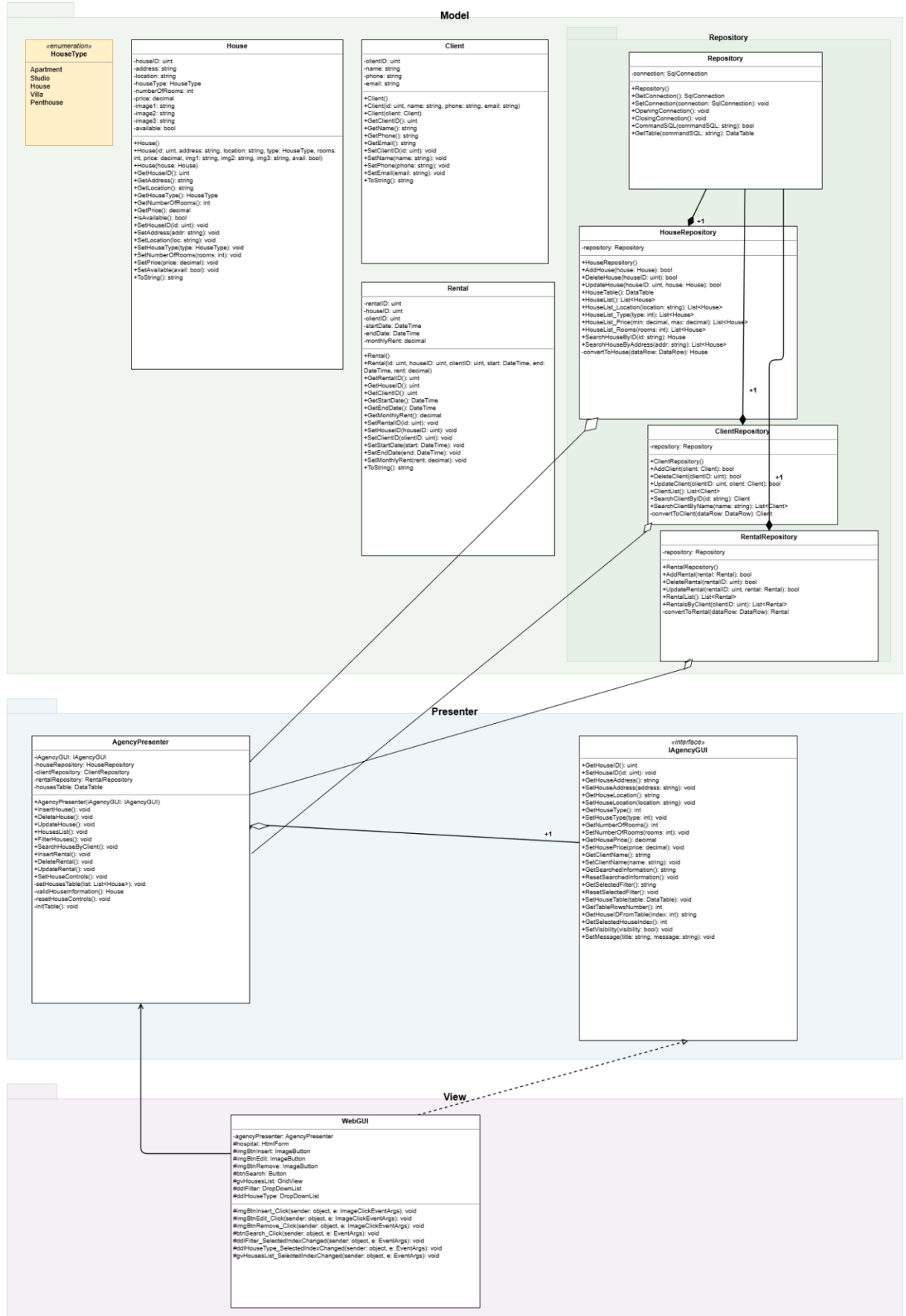
4.5 Diagrama de Clase

Diagrama de clase reflecta arhitectura MVP cu urmatoarele tipuri de relatii:

- Realizare (linie intrerupta cu sageata triunghiulara): HouseRepository realizeaza IHouseRepository; ClientRepository realizeaza IClientRepository; RentalRepository realizeaza IRentalRepository; JavaFXGUI realizeaza IAgencyGUI;

- Agregare (linie continua cu romb gol la sursa - AgencyPresenter): AgencyPresenter agregate IAgencyGUI - presenter-ul detine o referinta la interfata view-ului, insa cele doua pot exista independent;
- Agregare (linie continua cu romb gol la sursa - AgencyPresenter): AgencyPresenter agregate IHouseRepository, IClientRepository, IRentalRepository - dependenta de abstractiunile modelului, nu de implementari concrete;
- Dependenta (linie intrerupta): HouseRepository, ClientRepository, RentalRepository folosesc DatabaseConnection;

- Asociere: Rental refera House si Client prin campurile houseID si clientID ;



5. Descrierea Aplicatiei

5.1 Arhitectura MVP

- Model: House, Client, Rental si repository-urile lor encapsuleaza accesul la date prin interfețe. Toate query-urile folosesc PreparedStatement pentru securitate;
- View: JavaFXGUI implementeaza IAgencyGUI si contine exclusiv logica de UI (TabPane cu 3 tab-uri, formulare, tabele, imagini). Nu contine nicio logica de business;
- Presenter: AgencyPresenter coordoneaza comunicarea MVP. Primeste comenzi din View, valideaza datele, apeleaza Model si actualizeaza View-ul cu rezultatele.

5.2 Structura Interfetei Grafice

Interfata este organizata in trei tab-uri:

Tab 1 - Locuinte

- Formular cu campurile: ID (readonly), Adresa, Locatie, Tip (ComboBox), Nr. Camere, Pret;
- Panel imagini: 3 ImageView-uri care afiseaza imaginile locuintei selectate din tabel;
- Tabel cu lista tuturor locuintelor;
- Sectiune de cautare: gaseste toate locuintele inchiriate de un client specificat prin nume;
- Sectiune de filtrare: 4 criterii disponibile - Locatie, Pret (format min-max), Tip locuinta, Nr. camere.

Tab 2 - Clienti

- Formular cu campurile: ID (readonly), Nume, Telefon, Email;
- Tabel cu lista tuturor clientilor;
- Butoane CRUD: Adauga, Actualizeaza, Sterge.

Tab 3 - Contracte de Inchiriere

- Formular cu campurile: ID Contract (readonly), ID Locuinta, ID Client, Data Start (yyyy-MM-dd), Data End (optional, gol = contract activ), Chirie lunara;
- Tabel cu lista tuturor contractelor;
- Butoane CRUD: Adauga, Actualizeaza, Sterge;
- La adaugarea unui contract activ (fara Data End), locuinta este marcata automat ca indisponibila (Available = false). La actualizarea unui contract cu Data End completata, locuinta devine disponibila din nou.

5.3 Structura Bazei de Date

Schema SQL pentru crearea bazei de date:

```
CREATE DATABASE RentalAgencyDB;  
USE RentalAgencyDB;
```

```
CREATE TABLE House (
```

```

ID INT AUTO_INCREMENT PRIMARY KEY,
Address VARCHAR(255) NOT NULL,
Location VARCHAR(100) NOT NULL,
HouseType INT NOT NULL, -- 0=Apartment, 1=Studio, 2=House, 3=Villa, 4=Penthouse
NumberOfRooms INT NOT NULL,
Price DOUBLE NOT NULL,
Image1 VARCHAR(500),
Image2 VARCHAR(500),
Image3 VARCHAR(500),
Available BOOLEAN DEFAULT TRUE
);

```

```

CREATE TABLE Client (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(150) NOT NULL,
  Phone VARCHAR(20) NOT NULL,
  Email VARCHAR(150) NOT NULL
);

```

```

CREATE TABLE Rental (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  HouseID INT NOT NULL,
  ClientID INT NOT NULL,
  StartDate DATE NOT NULL,
  EndDate DATE, -- NULL = contract activ
  MonthlyRent DOUBLE NOT NULL,
  FOREIGN KEY (HouseID) REFERENCES House(ID),
  FOREIGN KEY (ClientID) REFERENCES Client(ID)
);

```

5.4 Principii SOLID Respectate

- Single Responsibility (SRP): HouseRepository gestioneaza doar accesul la date pentru House; JavaFXGUI gestioneaza exclusiv UI-ul; AgencyPresenter coordoneaza doar logica de prezentare;
- Open/Closed (OCP): adaugarea unui nou criteriu de filtrare sau a unui nou repository nu necesita modificarea claselor existente, ci extinderea lor;
- Liskov Substitution (LSP): HouseRepository, ClientRepository si RentalRepository pot substitui interfetele lor fara a afecta comportamentul aplicatiei;
- Interface Segregation (ISP): IHouseRepository, IClientRepository si IRentalRepository sunt interfete separate, specifice fiecarei entitati, cu metode relevante doar pentru acea entitate;
- Dependency Inversion (DIP): AgencyPresenter depinde exclusiv de abstractiuni - IAgencyGUI, IHouseRepository, IClientRepository, IRentalRepository - injectate prin constructor. Nicio dependenta de implementari concrete.

5.5 Securitate - Prevenirea SQL Injection

Toate operatiunile de acces la baza de date folosesc PreparedStatement in locul concatenarii de string-uri:

```

// GRESIT - vulnerabil la SQL Injection:
String sql = "SELECT * FROM House WHERE Location LIKE '%" + location + "%'";

```

```
// CORECT - PreparedStatement:  
String sql = "SELECT * FROM House WHERE Location LIKE ?";  
PreparedStatement ps = conn.prepareStatement(sql);  
ps.setString(1, "%" + location + "%");
```

De asemenea, toate resursele JDBC (Connection, PreparedStatement, ResultSet) sunt gestionate cu try-with-resources, eliminand riscul de memory leak.

6. Concluzii

Aplicatia implementeaza complet cerintele problemei 15, cu separare clara intre straturile MVP, respectarea principiilor SOLID si securizarea accesului la baza de date prin PreparedStatement.

Functionalitati implementate:

- CRUD complet pentru Locuinte, Clienti si Contracte de inchiriere;
- Afisarea imaginilor locuintelor la selectia din tabel;
- Filtrarea locuintelor dupa 4 criterii;
- Cautarea locuintelor inchiriate dupa numele clientului;
- Gestionarea automata a disponibilitatii locuintei la adaugarea/incheierea contractelor.